

Q: When is $\mathbb{Z}_n^*$ cyclic?

Ex:

• $\mathbb{Z}_8^* = \{1, 3, 5, 7\}$ has order 4 ($= \phi(8)$) but

$$1^2 = 3^2 = 5^2 = 7^2 = 1$$

in $\mathbb{Z}_8^*$. Therefore, each element of $\mathbb{Z}_8^*$ has order

≤ 2 and $\mathbb{Z}_8^*$ is not cyclic.

Note: a finite group $(G, *)$ is cyclic iff it contains an element of order $|G|$.

• $\mathbb{Z}_5^* = \{1, 2, 3, 4\} = \{2^0, 2^1, 2^2, 2^3\} = \langle 2 \rangle_{\mathbb{Z}_5^*}$
4 3

is cyclic.

Defⁿ An integer $a \in \mathbb{Z}$ is a primitive root modulo n if the following holds: for each $b \in \mathbb{Z}$ with $\gcd(b, n) = 1$ there exists an integer $k \geq 1$ with $b \equiv a^k \pmod{n}$.

(Note: such a number a will satisfy $\gcd(a, n) = 1$)

Fact: There exists a primitive root modulo n
if and only if $\mathbb{Z}_n^\times$ is cyclic.

More precisely, $a \in \mathbb{Z}$ is a primitive root modulo n
if and only if $\mathbb{Z}_n^\times = \langle a \bmod n \rangle_{\mathbb{Z}_n^\times}$.

Thm (Gauss)

$\mathbb{Z}_n^\times$ is cyclic if and only if $n = 1, 2, 4$ or n
is of the form $n = p^a$ or $n = 2p^a$ for an
odd prime p and integer $a \geq 1$.

Some problems that we can't get into:

Suppose that $\mathbb{Z}_n^\times$ is cyclic. How can we find a
generator? How big is the smallest generator?

(This gets deep.)

How many generators are there?

Discrete logarithms

Let G be a finite cyclic group of order n with generator g (so $G = \langle g \rangle_G = \{g^i : i = 0, \dots, n-1\}$).

Then for each $h \in G$, there exists a unique (!)

$a \in \{0, 1, \dots, n-1\}$ with $h = g^a$.

The number a is called the discrete logarithm of h to the base g . We sometimes write $a = \log_g^G(h)$.

The discrete logarithm problem asks, given (G, g, h) , to determine $a = \log_g^G(h)$.

Ex: • $\mathbb{Z}_9^\times = \langle 2 \rangle$ is cyclic of order 6.

We have $2^4 = 7$ in $\mathbb{Z}_9^\times$ and so $\log_2^{\mathbb{Z}_9^\times}(7) = 4$.

• In the additive group $(\mathbb{Z}_9, +_9)$, we have

$\mathbb{Z}_9 = \langle 2 \rangle$. Since $2 \cdot 8 \equiv 7 \pmod{9}$, $\log_2^{\mathbb{Z}_9}(7) = 8$.

For cryptographic applications, we are looking for a group $G = \langle g \rangle_G$ s.t.

① Multiplication (hence exponentiation) can be carried out efficiently.

② Discrete logarithms are hard to compute.

Non-example:

Consider the additive group $G = \mathbb{Z}_n$.

Exercise: $g \in \mathbb{Z}_n$ generates $\mathbb{Z}_n$ iff $\gcd(g, n) = 1$.

Suppose that $\mathbb{Z}_n = \langle g \rangle_{\mathbb{Z}_n}$. Let $h \in \mathbb{Z}_n$ be arbitrary.

(Note: for $a \in \mathbb{Z}$, the group-theoretic power g^a in $(\mathbb{Z}_n, +_n)$ is just the multiple $a \cdot g$.)

Since $\gcd(g, n) = 1$, $\exists \bar{g} \in \mathbb{Z}_n$: $g \bar{g} \equiv 1 \pmod{n}$.

Hence: $ag \equiv h \pmod{n}$ iff $a \equiv \bar{g}h \pmod{n}$.

We may thus effectively compute discrete logarithms in $\mathbb{Z}_n$ using the extended Euclidean algorithm.

Remark: • let p be a "large" prime and let $g \in \mathbb{Z}_p^\times$ with $\mathbb{Z}_p^\times = \langle g \rangle_{\mathbb{Z}_p^\times}$. (That is, g is a primitive root modulo p)

Then the discrete logarithm problem in $\mathbb{Z}_p^\times$ is believed to be generally very difficult.

(Some assumptions need to be made though. For instance, $p-1$ should have a "large" prime divisor. Otherwise, one can exploit the Chinese remainder theorem, "Hensel lifting", and other techniques related to integer factorization)

- For some clever algorithms for finding discrete logarithms, see Ch. 13 in Smart's book.

Diffie-Hellman Key exchange (1976)

Objective:

Alice and Bob wish to agree on a shared secret key which they will then use together with a symmetric cryptosystem.

We assume that they can only communicate over an insecure channel in the sense that eavesdroppers might listen in on their messages.

Can this be done?

The Diffie-Hellman Key exchange protocol:

- Alice and Bob first agree on a finite cyclic group G of order n with generator g .
(E.g. $G = \mathbb{Z}_p^*$, p prime)

- Alice secretly chooses a random integer

$$a \in \{1, \dots, n-1\}.$$

She then computes $A := g^a$ in $\mathbb{Q}$ and sends A to Bob.

- Bob secretly chooses a random integer $b \in \{1, \dots, n-1\}$, computes $B := g^b$ in $\mathbb{Q}$ and sends B to Alice.

- Alice receives B and computes

$$B^a = g^{ab}$$

in $\mathbb{Q}$ (without knowing $b!$).

- Bob receives A and computes

$$A^b = g^{ab}$$

in $\mathbb{Q}$ (without knowing $a!$).

- Their shared secret key is $\boxed{g^{ab}}.$

Ex: $G = \mathbb{Z}_{17}^\times$ of order $n = \phi(17) = 16$.

Fact/exercise: $G = \langle g \rangle_{\mathbb{Z}_{17}^\times}$ for $g = 3$.

- Alice chooses $a = 7$, computes $A = g^a \bmod 17 = 11$ and sends it to Bob.
- Bob chooses $b = 4$, computes $B = g^b \bmod 17 = 13$ and sends it to Alice.
- Alice receives $B = 13$ and computes $B^a \bmod 17 = 4$.
- Bob receives $A = 11$ and computes $A^b \bmod 17 = 4$.
- Their shared secret key is 4.

On the security of Diffie-Hellman Key exchange

Suppose that Eve intercepts $G, g, A (=g^a)$, and $B (=g^b)$.

If she can solve the discrete logarithm problem in G ,

she can determine $a = \log_g^G(A)$ and $b = \log_g^G(B)$

and hence compute the shared secret key g^{ab} .

Hence: breaking a Diffie-Hellman Key exchange is at
most as hard as finding discrete logarithms.

It is unknown if there is an easier way.

Diffie-Hellman Key exchange in practice

- Various common protocols use (or may use) Diffie-Hellman Key exchanges (eg. SSH, TLS, HTTPS)
- For $\mathbb{Z}_p^*$, p should have at least 2048 binary digits.
→ Log jam attack (public since 2015)

One can use the ideas of the Diffie-Hellman key exchange protocol to design a public-key cryptosystem.

The ElGamal cryptosystem (1985)

• Key generation:

Alice chooses a cyclic group G of order n and a generator g of G .

She chooses a random exponent $a \in \{1, \dots, n-1\}$ and

computes $A := g^a$ in G .

Her public key is (G, g, A) and her private key

is (G, g, a) .

• Encryption: Bob encrypts a message $m \in G$ as follows. First, he chooses a random $b \in \{1, \dots, n-1\}$ and computes $B = g^b$ in G . He then computes $c := A^b * m$ (he uses Alice's public key for this!)

The ciphertext corresponding to the plaintext m is (B, c) .

• Decryption: Alice receives $(B, c) = (g^b, g^{ab}m)$.

She computes $B^a = g^{ab}$ and recovers

$$m = (B^a)^{-1} * c.$$

Remark: One can show that the ElGamal cryptosystem is as secure as Diffie-Hellman key exchange.